DOCUMENT INTERNE

CAHIER DES CHARGES

LaraSgmefQR v3.0

Ameliorations du Package Laravel

Specification fonctionnelle et technique pour la mise en conformite DGI, la refonte architecturale et les nouvelles fonctionnalites du package de facturation electronique beninoise.

Equipe Technique | Juin 2025

Table des matieres

1. Introduction et Contexte 3

2. Etat Actuel du Package (v2.1.0) 3

2.1 Fonctionnalites existantes . . . 3

2.2 Architecture actuelle . . . 4

2.3 Limitations identifiees . . . 4

3. Exigences de Conformite DGI 5

3.1 Auto-declaration SFE . . . 5

3.2 Les 20 cas de test obligatoires . . . 6

3.3 Validation du QR Code . . . 7

3.4 Correspondance API e-MECeF . . . 7

3.5 Elements de securite obligatoires . . . 7

4. Ameliorations Architecturales 8

4.1 Refactoring des DTOs . . . 8

4.2 Support Multi-IFU . . . 8

4.3 Files d'attente pour les factures (Job Queue) . . . 9

4.4 Middleware de securite renforce . . . 9

4.5 Diagramme de sequence : creation et confirmation . . . 9

4.6 Design patterns a implementer . . . 10

5. Nouvelles Fonctionnalites 10

5.1 Gestion des factures d'avoir (FA/EA) . . . 11

5.2 Export STAT . . . 11

5.3 Gestion avancee des taxes . . . 12

5.4 Multi-paiement et Webhooks . . . 12

6. Ameliorations de l'Interface Utilisateur 12

6.1 Dashboard ameliore . . . 12

6.2 Design responsive et notifications . . . 13

6.3 Recherche et filtrage avances 13

7. Plan de Tests et Qualite 13

7.1 Tests unitaires 13

7.2 Tests d'integration et de conformite 14

7.3 Couverture cible et CI/CD 14

8. Planning et Livrables 14

1. Introduction et Contexte

Le systeme e-MECeF (electronic Mechanism for Centralized Invoicing) constitue la plateforme officielle de facturation electronique mise en place par la Direction Generale des Impots (DGI) du Benin. Ce dispositif reglementaire impose a tous les contribuables assujettis de produire des factures normalisees via un Systeme de Facturation Electronique (SFE) dument homologue. L'objectif principal de cette demarche est de renforcer la transparence des transactions commerciales, de lutter contre la fraude fiscale et d'ameliorer le recouvrement des impots sur le territoire national. Chaque facture emise doit etre verifiable en ligne via le portail de la DGI a l'adresse https://developper.impots.bj/sygmef-test/verification, garantissant ainsi l'authenticite et la conformite de chaque document fiscal.

Le package LaraSgmefQR, dans sa version actuelle 2.1.0, offre une integration Laravel avec l'API SyGM-eMCF. Il propose une architecture basee sur les principes SOLID, des DTOs pour le typage fort des donnees, un client HTTP robuste avec gestion des erreurs et des retry, une interface web de gestion des factures, un systeme d'evenements Laravel, et des templates de factures multi-formats. Ce package est utilise par plusieurs editeurs de SFE et contributeurs au Benin pour se conformer aux exigences de la DGI en matiere de facturation electronique.

Le present cahier des charges a pour objectif de guider le developpement de la version 3.0 du package LaraSgmefQR. Il couvre quatre axes principaux : la mise en conformite complete avec les exigences de la DGI, notamment le processus d'auto-declaration des SFE et les vingt cas de test obligatoires ; la refonte architecturale pour supporter le multi-IFU, les files d'attente asynchrones et les patterns de conception avances ; l'ajout de nouvelles fonctionnalites telles que la gestion des factures d'avoir, l'export STAT et le multi-paiement ; enfin, l'amelioration de l'experience utilisateur avec un dashboard moderne, des notifications en temps reel et un design responsive. Les documents de reference incluent la specification de l'API e-MECeF v1.0 et la procedure d'auto-declaration de SFE fournie par la DGI.

2. Etat Actuel du Package (v2.1.0)

2.1 Fonctionnalites existantes

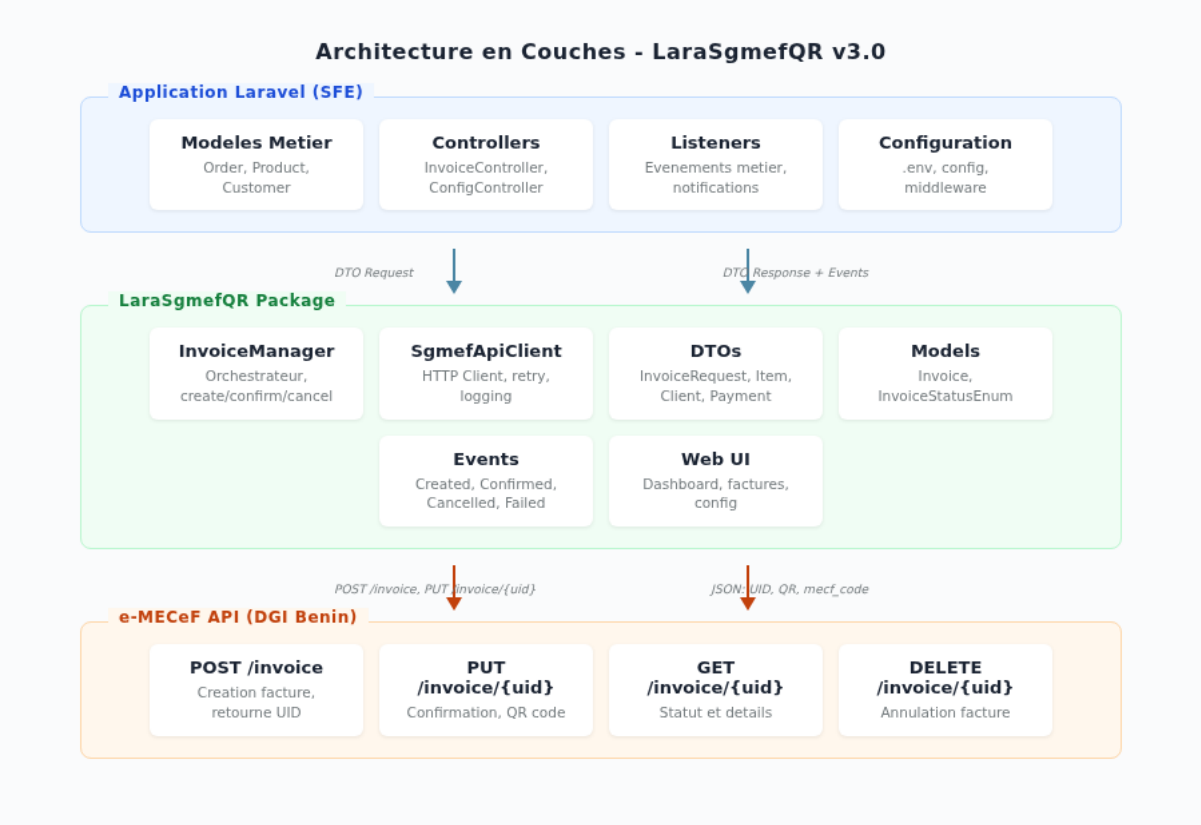
La version 2.1.0 de LaraSgmefQR propose un ensemble de fonctionnalites solides qui constitue la base du package. Le client API (SgmefApiClient) offre une communication HTTP robuste avec l'API e-MECeF, incluant la gestion des timeouts, le retry automatique en cas d'echec, le logging complet des requetes et reponses, ainsi qu'une validation stricte des donnees envoyees. L'architecture respecte les principes Clean Code et SOLID, avec des Data Transfer Objects (DTOs) pour le typage fort, des interfaces et contrats pour le decouplage, et l'injection de dependances pour l'inversion de controle. Le service principal, InvoiceManager, orchestre le cycle de vie complet des factures : creation, confirmation, annulation et synchronisation avec l'API distante.

L'interface web integree fournit un dashboard avec des statistiques en temps reel, une gestion complete des factures avec formulaire de creation, details, confirmation, annulation et telechargement PDF. Le package est totalement independant de tout systeme d'authentification, fonctionnant avec le concept

d'operateur configurable. Il est compatible avec tous les projets Laravel 10 et superieurs, et supporte PHP 8.1+. Un systeme d'evenements Laravel (InvoiceCreated, InvoiceConfirmed, InvoiceCancelled, InvoiceCreationFailed) permet une integration fluide avec l'application hote. Les exceptions personnalisees couvrent tous les cas d'erreur : InvoiceException, SgmefApiException, et leurs sous-classes specialisees.

2.2 Architecture actuelle

Le schema ci-dessous illustre l'architecture en trois couches du package. La couche superieure represente l'application Laravel (le SFE du contribuable), qui interagit avec le package LaraSgmefQR au milieu via les DTOs et les evenements. Le package lui-meme communique avec l'API e-MECeF de la DGI a travers son client HTTP specialise. Chaque composant est decouple grace aux interfaces et a l'injection de dependances, permettant une testabilite maximale et une grande flexibilite d'integration dans des projets existants.



2.3 Limitations identifiees

Malgre ses qualites, la version actuelle presente plusieurs limitations qui empechent une conformite totale avec les exigences de la DGI. Ces lacunes doivent etre adressees en priorite dans la version 3.0 afin de permettre aux editeurs de SFE d'obtenir l'approbation de la DGI lors du processus d'auto-declaration. Les principales limitations sont les suivantes :

- Absence du workflow d'auto-declaration SFE : le package ne guide pas l'utilisateur a travers le processus complet de verification et de soumission des Annexe 1 et 2 a la DGI (emecefbenin@finances.bj).
- Pas de support pour les factures d'avoir (FA/EA) : le champ **reference** permettant de lier une facture d'avoir a sa facture d'origine n'est pas implemente dans l'InvoiceRequestDto.
- Gestion mono-IFU uniquement : un seul IFU est configurable via SGMEF_DEFAULT_IFU, ce qui ne permet pas la gestion multi-etablissements pour les entreprises disposant de plusieurs points de vente.
- Champs manquants dans InvoiceItemDto : les proprietes **taxSpecific**, **originalPrice** et **priceModification** requises par l'API e-MECeF pour certains scenarios de facturation ne sont pas presentes.
- Aucun export STAT : la generation de fichiers recapitulatifs mensuels ou periodiques necessaires au reporting fiscal aupres de la DGI n'est pas implementee.
- Support limite des types de paiement : seuls ESPECES, CARTEBANCAIRE, CHEQUE et VIREMENT sont documentes, alors que l'API supporte egalement MOBILEMONEY, CREDIT et AUTRE.
- Dashboard sans notifications temps reel : les statistiques sont calculees a chaque chargement de page sans capacite de rafraichissement automatique ou de notifications push.
- Pas de nettoyage automatique des factures en attente : les factures non confirmees dans le delai de 2 minutes expirent cote e-MECeF mais ne sont pas nettoyees automatiquement dans la base locale.

3. Exigences de Conformite DGI

Ce chapitre constitue le coeur du cahier des charges. La mise en conformite avec les exigences de la Direction Generale des Impots est la priorite absolue de la version 3.0. Sans cette conformite, aucun editeur de SFE ne peut obtenir l'approbation necessaire a l'utilisation du package en production. Chaque sous-section detaille un aspect specifique des exigences reglementaires et les adaptations techniques requises au niveau du package.

3.1 Auto-declaration SFE

La procedure d'auto-declaration est obligatoire pour tout editeur de SFE ou contribuable utilisant son propre systeme de facturation. Elle vise a demontrer a la DGI que le SFE produit des factures conformes aux specifications techniques de l'e-MECeF. Le processus se decompose en plusieurs etapes sequentielles que le package doit guider automatiquement.

- **Acces a la plateforme e-MECeF TEST** : obtenir les identifiants de test aupres de la DGI pour acceder a l'environnement de test a l'adresse https://developper.impots.bj/sygmef-emcf. Le package doit pouvoir basculer entre les environnements TEST et PRODUCTION via un parametre de configuration unique.
- **Modification du SFE selon la documentation** : mettre en conformite toutes les sorties de factures (PDF, impressions) avec les elements obligatoires : montants identiques, articles identiques, elements

de securite avec etiquettes et valeurs correctes, code QR obligatoire et scannable.

- **Verification des factures sur le portail DGI** : chaque facture generee doit etre verifiable a l'adresse https://developper.impots.bj/sygmef-test/verification. Le format de la facture imprimee peut differer du format du serveur, mais tous les elements (montants, articles, securite, QR) doivent etre strictement identiques.
- **Remplissage des Annexe 1 et 2** : le formulaire d'auto-declaration contient des cas de test pour divers types de factures et scenarios. Pour chaque cas, une capture d'ecran de la facture doit etre fournie avec toutes les donnees visibles et le QR code scannable.
- **Soumission a emecefbenin@finances.bj** : le dossier complet est envoye par email a la DGI pour validation. Si des problemes sont detectes, des mesures correctives doivent etre appliquees et le processus repete jusqu'a obtention de l'approbation.

3.2 Les 20 cas de test obligatoires

L'auto-declaration requiert la demonstration de vingt cas de test couvrant l'ensemble des types de factures, groupes de taxes et scenarios de paiement. Le tableau ci-dessous presente les quinze cas de test prioritaires que le package doit imperativement supporter. Les cinq cas restants devront etre ajoutes en fonction du secteur d'activite specifique de l'editeur. Chaque cas de test doit etre automatise dans la suite de tests du package pour garantir la non-regression.

No	Type	Groupe Taxe	Scenario de Test	Priorite
1	FV	A	Facture de vente avec un seul article exonere (groupe A, taux 0%)	Haute
2	FV	B	Facture de vente avec un seul article TVA 18% (groupe B)	Haute
3	FV	C	Facture de vente avec article TVA 18% + AIB 1% (groupe C)	Haute
4	FV	D	Facture de vente avec article TVA 18% + AIB 5% (groupe D)	Haute
5	FV	B+C	Facture multi-articles avec groupes de taxe mixtes (B et C)	Haute
6	FV	A+F	Facture avec article exonere et article exportation (0%)	Moyenne
7	FA	B	Facture d'avoir avec reference a une facture de vente originale	Haute
8	FA	C	Facture d'avoir avec AIB, reference obligatoire a la facture initiale	Haute
9	EV	B	Export de facture de vente (type EV) pour declaration	Haute
10	EA	B	Export de facture d'avoir (type EA) pour declaration	Haute
11	FV	B	Facture avec paiement multiple (ESPECES + CARTEBANCAIRE)	Haute
12	FV	B	Facture avec paiement MOBILEMONEY	Moyenne
13	FV	B	Facture avec taxe specifique (taxSpecific) sur un article	Haute
14	FV	B	Facture avec priceModification (remise ou majoration unitaire)	Haute
15	FV	B	Facture avec multi-paiement VIREMENT + CHEQUES + CREDIT	Moyenne

Pour les cas de test 16 a 20, l'editeur de SFE doit fournir des exemples supplementaires lies a son secteur d'activite specifique (par exemple : factures de services, factures avec remise globale, factures de cantiene, factures avec exoneration partielle). Si un cas de test parmi les 20 n'est pas applicable a l'activite de l'editeur, celui-ci doit explicitement indiquer : "Notre SFE n'utilise pas ce type de facture". Dans ce cas, le SFE ne sera pas autorise a produire ces types de factures apres approbation.

3.3 Validation du QR Code

Le code QR est un element de securite obligatoire de toute facture electronique confirmee. Il est genere par l'API e-MECeF lors de l'etape de confirmation (PUT /invoice/{uid}) et retourne dans le champ qr_code_data de la reponse. Ce QR code doit etre imprime sur chaque facture et scannable par tout lecteur QR standard. La verification s'effectue sur le portail de la DGI a l'adresse https://developper.impots.bj/sygmef-test/verification. Le package doit integrer un utilitaire de validation QR qui verifie automatiquement que les donnees encodees dans le QR correspondent aux elements de la facture (UID, montant, type). De plus, le package doit generer des images QR de haute resolution (minimum 300 DPI) pour garantir la lisibilite sur les impressions papier.

3.4 Correspondance API e-MECeF

Le tableau suivant dresse la correspondance entre les endpoints de l'API e-MECeF v1.0 et l'implementation actuelle du package. Cette analyse permet d'identifier les manquements et les actions correctives necessaires pour atteindre une couverture complete de l'API. Chaque endpoint est evalue en fonction de son statut d'implementation et de l'action requise pour la version 3.0.

Endpoint	Description	Statut Actuel	Action Requise
POST /invoice	Creation de facture	Implemente	Ajouter taxSpecific, originalPrice, priceModification dans les items
PUT /invoice/{uid}	Confirmation de facture	Implemente	Verifier la gestion complete des elements de securite
GET /invoice/{uid}	Recuperation statut	Implemente	Ajouter le support de la synchronisation incrementale
DELETE /invoice/{uid}	Annulation de facture	Implemente	Verifier la coherence avec l'enum InvoiceStatusEnum
POST /invoice/cancel	Annulation alternative	Non implemente	Ajouter le support de cette methode alternative
GET /tax-groups	Liste groupes de taxe	Non implemente	Implementer pour validation dynamique des groupes
GET /verification	Verification facture	Non implemente	Integrer la verification automatique post-confirmation

3.5 Elements de securite obligatoires

Lors de la confirmation d'une facture, l'API e-MECeF retourne un ensemble d'elements de securite qui doivent etre strictement reproduits sur la facture imprimee. Le package doit stocker et exposer ces elements via le DTO InvoiceResponseDto et les inclure dans les templates de facture PDF. L'absence d'un seul de ces elements constitue un motif de rejet lors de l'auto-declaration. Les elements de securite suivants sont obligatoires :

- **mecf_code** : le code MECeF officiel attribue par la DGI, identifiant unique de la facture dans le systeme fiscal.
- **qr_code_data** : les donnees brutes encodees dans le QR code, permettant la verification en ligne de la facture.
- Toutes les etiquettes de securite retournees dans l'objet SecurityElementsDto de la reponse API, incluant notamment le numero de controle, le hash de la facture, et les signatures numeriques.
- La reproduction exacte des etiquettes (labels) et de leurs valeurs correspondantes telles que retournees par l'API.

4. Ameliorations Architecturales

La version 3.0 du package necessite une refonte architecturale significative pour repondre aux exigences de conformite DGI tout en ameliorant la maintenabilite, l'extensibilite et les performances du code. Ce chapitre detaille les modifications structurelles planifiees, depuis le refactoring des DTOs jusqu'a l'implementation de patterns de conception avances.

4.1 Refactoring des DTOs

Les Data Transfer Objects constituent la couche de donnees fondamentale du package. L'analyse de l'API e-MECeF v1.0 a revele plusieurs champs manquants dans l'InvoiceItemDto actuel. Ces champs sont necessaires pour couvrir l'ensemble des cas de test obligatoires de la DGI. Le refactoring doit ajouter les proprietes suivantes a InvoiceItemDto :

```
class InvoiceItemDto { public string $name; public float $price; public int $quantity;
public string $taxGroup; // A, B, C, D, E, F public ?string $code; public ?string
$description; // NOUVEAUX CHAMPS v3.0 public ?float $taxSpecific; // Montant taxe
specifique public ?float $originalPrice; // Prix original avant modification public
?float $priceModification; // Remise ou majoration public ?string $reference; //
Reference article externe }
```

La methode validate() de chaque DTO doit etre enrichie pour verifier la coherence des nouvelles proprietes. Par exemple, si taxSpecific est defini, il doit etre positif et inferieur au prix unitaire. Si priceModification est present, originalPrice doit egalement etre renseigne pour permettre le calcul du prix final. Ces validations garantiront que les donnees envoyees a l'API e-MECeF sont conformes des la premiere requete, evitant les rejets et les allers-retours.

4.2 Support Multi-IFU

Les entreprises beninoises disposant de plusieurs etablissements ou succursales doivent pouvoir emettre des factures avec des IFU differents depuis une meme application Laravel. Le package actuel ne supporte

qu'un seul IFU par installation via la variable d'environnement SGMEF_DEFAULT_IFU. La version 3.0 introduira une architecture multi-IFU basee sur un resolver d'interface :

```
interface IfuResolverInterface { /** * Resoudre l'IFU a utiliser pour un contexte
donne. */ public function resolve( ?string $context = null ): string; } //
Implementation par defaut : IFU unique class DefaultIfuResolver implements
IfuResolverInterface { public function resolve(?string $ctx = null): string { return
config('lara_sgmef_qr.company.default_ifu'); } }
```

Cette approche par interface permet a chaque application hote de fournir sa propre logique de resolution IFU, que ce soit basee sur l'etablissement de l'utilisateur connecte, sur un parametre de requete, ou sur toute autre regle metier. Le binding Laravel sera effectue dans le ServiceProvider du package avec une portee scoped, permettant a differentes parties de l'application d'utiliser des IFU differents simultanement.

4.3 Files d'attente pour les factures (Job Queue)

La creation et la confirmation de factures via l'API e-MECeF impliquent des appels réseau qui peuvent prendre plusieurs secondes. Pour eviter de bloquer les requetes HTTP de l'utilisateur et garantir une bonne experience, le package v3.0 integrera un systeme de files d'attente Laravel. Deux jobs principaux seront implementes : CreateInvoiceJob pour la creation asynchrone et ConfirmInvoiceJob pour la confirmation. Le delai d'expiration de 2 minutes impose par l'e-MECeF sera gere automatiquement par un job programme de nettoyage des factures en attente expirees.

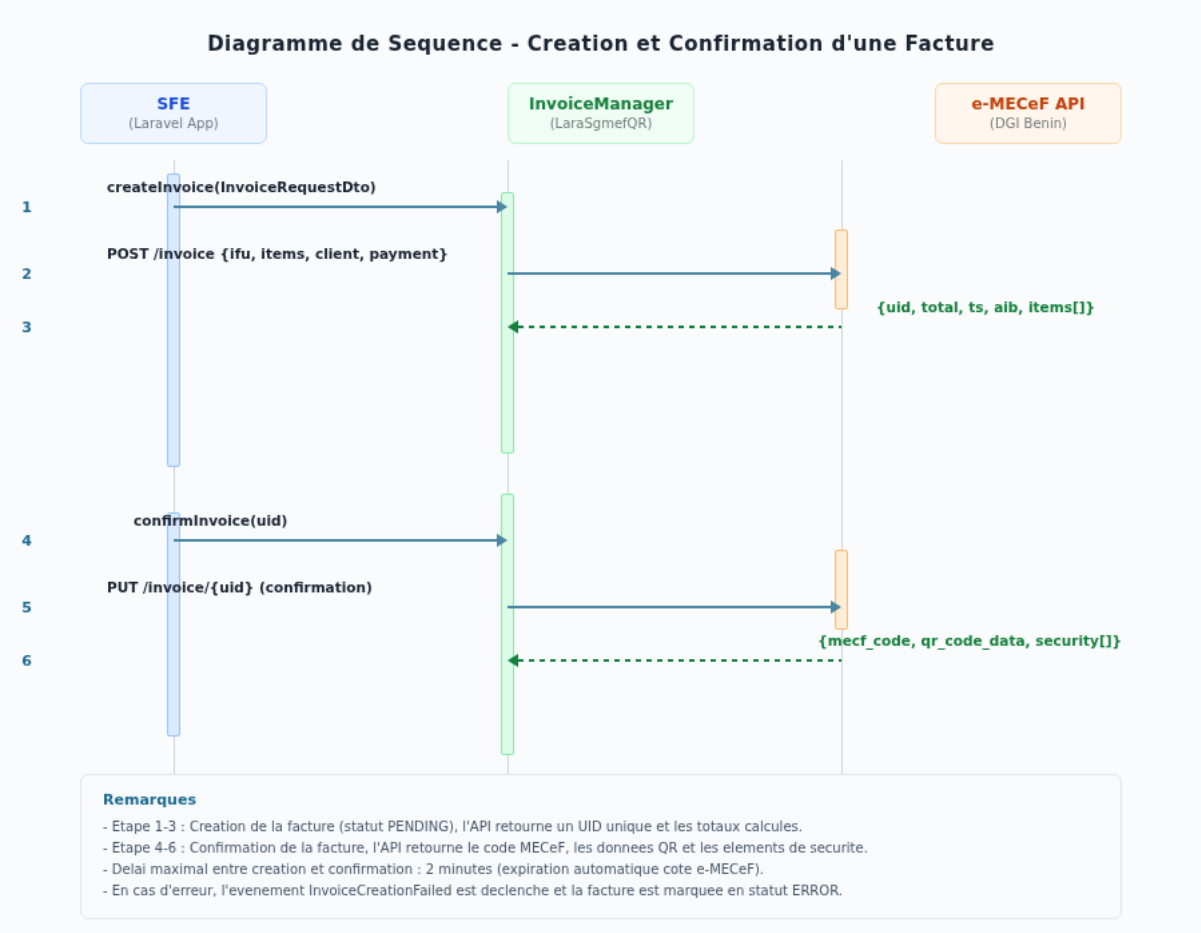
```
class CreateInvoiceJob implements ShouldQueue { use Dispatchable, InteractsWithQueue,
Queueable, SerializesModels; public function __construct( public InvoiceRequestDto
$data ) {} public function handle( InvoiceManagerInterface $manager ): Invoice {
return $manager->createInvoice($this->data); } }
```

4.4 Middleware de securite renforce

La securite des echanges entre le SFE et l'API e-MECeF sera renforcee par l'ajout d'un middleware de securite au niveau du client HTTP. Ce middleware implementera plusieurs mecanismes de protection : le rate limiting pour eviter de saturer l'API en cas de pic de facturation, la validation systematique du token JWT avant chaque requete, et la signature des requetes pour garantir l'integrite des donnees en transit. En cas d'echec d'authentification, le middleware declenchera automatiquement un evenement d'alerte pour permettre a l'application hote de reagir rapidement (notification administrateur, basculement vers un mode degrade, etc.).

4.5 Diagramme de sequence : creation et confirmation

Le diagramme de sequence ci-dessous illustre le flux complet de creation et de confirmation d'une facture. Ce processus en deux etapes est le coeur du fonctionnement du package : la premiere requete (POST /invoice) cree la facture et obtient un UID unique avec les totaux calcules, puis la seconde requete (PUT /invoice/{uid}) confirme la facture et recupere les elements de securite et le QR code. Le delai maximal entre les deux etapes est de 2 minutes, apres quoi la facture non confirmee expire cote serveur e-MECeF.



4.6 Design patterns a implementer

Pour ameliorer la maintenabilite et l'extensibilite du package, trois patterns de conception seront systematiquement appliques dans la version 3.0. Le Repository Pattern sera utilise pour encapsuler l'acès aux donnees des factures, permettant de facilement changer la source de stockage (MySQL, PostgreSQL, ou meme un stockage externe) sans modifier la logique metier. Le Strategy Pattern sera applique aux calculs de taxes, chaque groupe de taxe (A a F) etant implemente comme une strategie independante, facilitant l'ajout de nouveaux groupes ou la modification des regles de calcul existantes. Enfin, le Observer Pattern, deja partiellement implemente via les evenements Laravel, sera formalise avec des listeners pour le nettoyage automatique, les notifications et le monitoring.

5. Nouvelles Fonctionnalites

Au-dela de la mise en conformite, la version 3.0 introduit de nouvelles fonctionnalites qui repondent aux besoins concrets des editeurs de SFE et des contributeurs. Le tableau ci-dessous synthetise l'ensemble des nouvelles fonctionnalites avec leur priorite, leur complexite de developpement et leurs dependances.

Fonctionnalite	Description	Priorite	Complexite	Dependances
Factures d'avoir (FA/EA)	Gestion complete avec reference a la facture d'origine	Haute	Elevee	DTOs refactoring
Export STAT	Recapitulatifs mensuels/periodiques pour le reporting DGI	Haute	Moyenne	Modele Invoice
Calcul avance des taxes	Calcul automatique par groupe, apercu avant soumission	Haute	Elevee	Strategy Pattern
Multi-paiement	Support de tous les types : ESPECES, VIREMENT, CARTEBANCAIRE, MOBILEMONEY, CHEQUES, CREDIT, AUTRE	Moyenne	Moyenne	PaymentDto
Webhooks/Notifications	Systeme de notification pour les changements de statut	Moyenne	Moyenne	Events existants
Nettoyage automatique	Suppression des factures en attente expirees (2 min)	Haute	Faible	Job Queue
Verification QR integree	Validation automatique du QR code post-confirmation	Haute	Moyenne	API verification
Mode multi-IFU	Gestion de plusieurs etablissements avec IFU distincts	Moyenne	Elevee	IfuResolverInterface

5.1 Gestion des factures d'avoir (FA/EA)

Les factures d'avoir (FA pour les factures d'avoir de vente, EA pour les exports d'avoir) permettent de corriger ou d'annuler partiellement une facture de vente initiale. L'implementation dans la version 3.0 necessite l'ajout du champ reference dans l'InvoiceRequestDto. Ce champ obligatoire contient l'UID de la facture de vente originale a laquelle l'avoir se rapporte. L'API e-MECeF effectue automatiquement les verifications de coherence entre la facture d'avoir et la facture originale (montant de l'avoir inferieur ou egal au montant initial, meme IFU, etc.). Le package doit valider ces regles en local avant l'envoi a l'API pour fournir des messages d'erreur clairs a l'utilisateur.

```
$creditNote = InvoiceRequestDto::from([ 'ifu' => $resolver->resolve($etablissement), 'type' => 'FA', 'reference' => $originalInvoice->uid, 'client' => [ 'name' => $originalInvoice->client_name, 'ifu' => $originalInvoice->customer_ifu, ], 'items' => $returnItems, 'payment' => [['name' => 'VIREMENT', 'amount' => $returnTotal]], ]);
```

5.2 Export STAT

L'export STAT est un fichier recapitulatif periodique des factures emises, necessaire au reporting fiscal aupres de la DGI. Le package v3.0 integrera une commande Artisan et un service dedie pour generer ces exports. Le format de sortie sera configurable (CSV, XLSX, PDF) et inclura les informations suivantes : periode de reference, nombre total de factures, montant total HT et TTC, montant total des taxes collectees, repartition par type de facture (FV, FA, EV, EA), et repartition par groupe de taxe. L'export pourra etre genere manuellement via la commande Artisan ou automatiquement via un job programme

mensuel.

5.3 Gestion avancee des taxes

Le systeme de calcul des taxes sera refondu selon le pattern Strategy. Chaque groupe de taxe (A a F) sera implemente comme une classe independante implementant une interface TaxCalculationStrategyInterface. Cela permettra de calculer les taxes localement avant l'envoi a l'API, offrant ainsi un apercu precis des montants TTC a l'utilisateur lors de la saisie de la facture. Le tableau ci-dessous resume les taux appliques a chaque groupe de taxe conformement aux specifications de la DGI.

Groupe	Description	TVA	AIB	Taux Total
A	Exonere	0%	0%	0%
B	TVA standard	18%	0%	18%
C	TVA + AIB 1%	18%	1%	19%
D	TVA + AIB 5%	18%	5%	23%
E	Regime special	Variable	Variable	Variable
F	Exportation	0%	0%	0%

5.4 Multi-paiement et Webhooks

Le support complet des types de paiement est essentiel pour couvrir les usages courants au Benin. La version 3.0 documentation et valide les sept types de paiement suivants : ESPECES (paiement en especes), VIREMENT (virement bancaire), CARTEBANCAIRE (carte de credit ou debit), MOBILEMONEY (paiement mobile, tres repandu au Benin via les plateformes comme MTN MoMo et Moov Money), CHEQUES (cheque bancaire), CREDIT (paiement a credit avec echeancier), et AUTRE (tout autre mode de paiement). Le systeme de webhooks permettra aux applications hote de s'abonner aux changements de statut des factures et de declencher des actions automatiques (envoi d'email, mise a jour du stock, synchronisation avec un ERP tiers, etc.). Les webhooks seront configurables via un systeme d'events Laravel avec support de queues pour un traitement asynchrone.

6. Ameliorations de l'Interface Utilisateur

L'experience utilisateur est un facteur determinant pour l'adoption du package par les editeurs de SFE et leurs clients finaux. La version 3.0 met l'accent sur un dashboard informatif, un design responsive adapte aux terminaux mobiles, des notifications en temps reel, et des fonctionnalites avancees de recherche et de filtrage.

6.1 Dashboard ameliore

Le dashboard sera entierement repense pour offrir une vue synthetique et actionnable de l'activite de facturation. Des cartes KPI seront affichees en haut de page avec les indicateurs suivants : nombre de factures creees aujourd'hui, cette semaine et ce mois ; taux d'erreur des appels API ; temps moyen de reponse de l'API e-MECeF ; et nombre de factures en attente de confirmation. Des graphiques chronologiques permettront de visualiser les tendances sur les 30 derniers jours. Un systeme d'alertes visuelles signalera les anomalies (pic d'erreurs, factures expirees, seuils de volume depasses). Chaque carte KPI sera cliquable pour accéder au detail correspondant.

6.2 Design responsive et notifications

L'interface web adoptera une approche mobile-first avec un systeme de grille CSS flexible. Les formulaires de creation de facture seront optimises pour la saisie tactile avec des champs de taille adequate, des selecteurs de date natifs et des boutons d'action accessibles. Le mode sombre (dark mode) sera supporte via la propriete prefers-color-scheme du navigateur et un toggle manuel dans la barre de navigation. Les preferences de l'utilisateur seront persistees dans le localStorage du navigateur. Pour les notifications temps reel, deux approches seront proposees : le polling AJAX pour les installations ne disposant pas de WebSocket (verification toutes les 30 secondes), et le broadcasting Laravel Echo via Pusher ou Reverb pour les installations avances. Les notifications couvriront les evenements de creation, confirmation, echec et expiration des factures.

6.3 Recherche et filtrage avances

La page de liste des factures sera enrichie avec un systeme de filtrage multi-criteres complet. Les filtres disponibles incluront : plage de dates de creation, statut de la facture (pending, confirmed, cancelled, error), type de facture (FV, FA, EV, EA), nom ou IFU du client, et tranche de montant. Les filtres seront combinables et persistants dans l'URL pour permettre le partage de vues filtrees. Un systeme de tri par colonnes sera egalement ajoute sur tous les champs pertinents (date, montant, statut, type). La recherche?? (full-text) sera implementee sur le nom du client et la reference interne de la facture.

7. Plan de Tests et Qualite

La qualite du package est un engagement fondamental. La version 3.0 cible une couverture de tests de 95% minimum, avec une attention particuliere portee aux tests de conformite DGI. Le plan de tests est structure en quatre niveaux : tests unitaires, tests d'integration, tests de conformite DGI automatises, et tests de bout en bout.

7.1 Tests unitaires

Les tests unitaires couvriront l'ensemble des composants isolables du package : validation des DTOs (verifier que chaque DTO rejette les donnees invalides et accepte les donnees conformes), calculs de taxes par groupe (verifier les taux de TVA et AIB pour chaque groupe de A a F), validation du format IFU (exactement 13 chiffres), et logique metier de l'InvoiceManager (gestion des transitions de statut, verification des preconditions pour la confirmation et l'annulation). Chaque test sera independant et reproductible, sans dependance a un environnement externe. Les mocks seront utilises pour simuler les

dependances (client API, repository).

7.2 Tests d'integration et de conformite

Les tests d'integration verifieront le fonctionnement correct de l'ensemble du flux de facturation, du client HTTP jusqu'au stockage en base de donnees. Le client API sera mocke using les reponses typiques de l'e-MECeF pour simuler les cas nominaux et les cas d'erreur. Les tests de conformite DGI reprendront les vingt cas de test obligatoires de l'auto-declaration. Chaque cas sera automatise avec une fixture contenant les donnees d'entree attendues et les assertions sur la reponse de l'API mockee (presence de l'UID, exactitude des totaux, presence des elements de securite et du QR code). La validation QR sera egalement testee automatiquement en verifiant que les donnees du QR correspondent aux elements de la facture.

7.3 Couverture cible et CI/CD

L'objectif de couverture de tests est fixe a 95% minimum pour l'ensemble du code source du package. Un pipeline CI/CD sera configure avec GitHub Actions pour executer automatiquement les tests a chaque push et pull request. Le pipeline comprendra trois jobs principaux : le job de tests unitaires avec rapport de couverture, le job de tests d'integration avec un environnement de test dedie, et le job de deploiement sur l'environnement de staging apres validation. Le tableau suivant resume la strategie de tests.

Type de Test	Outil	Cible	Frequence
Tests unitaires	Pest / PHPUnit	DTOs, Taxes, Validation IFU	A chaque commit
Tests d'integration	Pest + HTTP Mocks	Flux complet facturation	A chaque commit
Conformite DGI	Pest + Fixtures	20 cas de test auto-declaration	Quotidien + pre-release
Couverture	PHPUnit --coverage	Objectif 95%+	A chaque commit
CI/CD	GitHub Actions	Test / Coverage / Stage	Automatique

8. Planning et Livrables

Le developpement de la version 3.0 sera organise en cinq phases progressives d'une duree de deux semaines chacune, soit un total de dix semaines de developpement. Chaque phase produit des livrables tangibles et verifiables, et se termine par des criteres de validation stricts qui doivent etre satisfaits avant de passer a la phase suivante. Ce decoupage permet un suivi regulier de l'avancement et une livraison incrementale de valeur.

Phase	Periode	Livrables	Criteres de Validation
Phase 1	Semaines 1-2	Refactoring des DTOs, ajout des champs manquants (taxSpecific, originalPrice, priceModification), ecriture des tests unitaires de validation des DTOs	Tous les DTOs valides, couverture DTOs > 95%

Phase	Periode	Livrables	Criteres de Validation
Phase 2	Semaines 3-4	Support Multi-IFU avec IfuResolverInterface, gestion complete des factures d'avoir (FA/EA) avec reference, extension des types de paiement	Tests d'integration multi-IFU et factures d'avoir passants
Phase 3	Semaines 5-6	Automatisation des 20 cas de test de conformite DGI, validation QR integree, nettoyage automatique des factures expirees	20 cas de test DGI automatises et passants, QR valide
Phase 4	Semaines 7-8	Dashboard ameliore avec KPIs, notifications temps reel, design responsive, mode sombre, recherche et filtrage avances	Interface fonctionnelle et testee sur mobile et desktop
Phase 5	Semaines 9-10	Export STAT, pipeline CI/CD, documentation complete, preparation du dossier d'auto-declaration, release candidate v3.0.0	Pipeline CI/CD operationnel, documentation complete, RC valide

A l'issue de la Phase 5, le package sera en etat de release candidate. Le dossier d'auto-declaration complet, comprenant les captures d'ecran des vingt cas de test, les Annexe 1 et 2 remplies, sera soumis a la DGI a l'adresse emecefbenin@finances.bj. Le package restera en version release candidate jusqu'a l'obtention de l'approbation formelle de la DGI, moment auquel la version 3.0.0 sera publiee officiellement. En cas de retour correctif de la DGI, les modifications seront appliquees dans une version 3.0.x et une nouvelle soumission sera effectuee dans les plus brefs delais.